

Learning a Lightweight Adaptive Retrieval Policy: Balancing Answer Quality, Faithfulness, and Retrieval Cost in Retrieval-Augmented Generation

Muhammad Maroof

Affiliation to be added
email@example.com

Abstract

Standard Retrieval-Augmented Generation (RAG) retrieves a fixed number of documents for every query, regardless of whether the underlying language model already knows the answer or whether a single retrieval pass is sufficient. This is computationally wasteful and can even hurt answer quality when irrelevant context is injected unnecessarily. Prior work has proposed adaptive alternatives: Self-RAG [Asai et al., 2024] fine-tunes the generator to emit retrieval-control tokens; Adaptive-RAG [Jeong et al., 2024] routes queries via a trained query-complexity classifier; FLARE [Jiang et al., 2023] triggers retrieval mid-generation based on token-level confidence; and Corrective RAG [Yan et al., 2024] evaluates retrieved evidence *after* an initial retrieval and corrects course if it is poor. We ask a narrower question than “can retrieval be adaptive” – which these works already answer affirmatively – and instead ask: *can a lightweight, non-fine-tuning controller, combining retrieval-confidence, evidence-coverage, and answer-confidence signals, learn a retrieval policy that matches or improves on existing adaptive RAG methods’ quality-faithfulness-cost trade-off, without fine-tuning the generator LLM?* We present the controller design, its mathematical formulation, and a pre-registered evaluation protocol – an accuracy/fairness-versus-cost Pareto analysis with bootstrap confidence intervals, paired significance testing, and a signal-ablation study – intended to test this question rigorously rather than assume it. *Consistent with the project’s research-integrity requirements, this draft reports no empirical results: Sections 6–8 are structural placeholders to be completed once the experiments in Section 5 are actually run.*

1 Introduction

Retrieval-Augmented Generation (RAG) augments a language model with a non-parametric memory – typically a dense retriever over a text corpus – to reduce hallucination and improve factual grounding on knowledge-intensive tasks [Lewis et al., 2020]. In its standard form, RAG retrieves the same number of documents k for every input query, irrespective of whether the query is easily answerable from the model’s parametric knowledge, whether it requires external evidence at all, or whether a single retrieval pass is even sufficient for multi-hop reasoning.

This “always-retrieve” policy has two costs. First, an *efficiency* cost: every query pays for embedding, similarity search, and a longer context window, even when unnecessary. Second, a *quality* cost: indiscriminately retrieving and incorporating passages regardless of necessity or relevance can

diminish the model’s own versatility and lead to unhelpful or even actively misleading generations [Asai et al., 2024].

A growing line of work makes retrieval *adaptive*. Mallen et al. [Mallen et al., 2023] were, to our knowledge, the first to study per-query retrieve-or-not decisions systematically, showing on their PopQA benchmark that unassisted language models remain competitive on high-popularity entities while retrieval augmentation helps substantially on long-tail knowledge. Self-RAG [Asai et al., 2024] internalizes the decision into the generator itself via fine-tuning on reflection tokens. FLARE [Jiang et al., 2023] makes the decision continuously during long-form generation based on token-level confidence. Adaptive-RAG [Jeong et al., 2024] trains an external classifier to route queries between no-retrieval, single-step, and iterative retrieval-augmented strategies based on predicted query complexity. Corrective RAG [Yan et al., 2024] always retrieves first, then uses a lightweight evaluator to decide whether to trust, discard, or supplement the retrieved evidence.

Research question. Given that ”can retrieval be adaptive” is already answered, we ask a narrower and more specific question:

Can a lightweight controller learn an optimal retrieval policy that balances answer quality, faithfulness, and retrieval cost better than existing adaptive RAG methods?

We take ”existing adaptive RAG methods” to mean, concretely, Adaptive-RAG and Corrective RAG as directly reproducible baselines (Self-RAG and FLARE are included as related work but are harder to reproduce fairly without fine-tuning infrastructure; see Section 5.2). We take ”lightweight” to specifically exclude generator fine-tuning, given realistic compute constraints (a single free-tier GPU).

Contributions.

1. A two-stage adaptive retrieval controller (cheap rule-based prefilter → lightweight classifier on ambiguous cases) that requires no generator fine-tuning (Section 4).
2. A pre-registered evaluation protocol that treats efficiency (retrieval calls, tokens, latency, estimated cost) as a first-class, statistically-tested axis alongside answer quality and faithfulness, rather than a secondary bullet point (Section 5).
3. A signal-ablation design that isolates which of retrieval-confidence, evidence-coverage, and answer-confidence actually drive good retrieval decisions (Section 5).
4. A fully reproducible, open implementation and configuration for all reported conditions.

No results are claimed in this draft. Every empirical statement in Sections 6–9 is explicitly marked pending and must be experimentally validated before publication.

2 Related Work

2.1 Retrieval-Augmented Generation

RAG combines a pre-trained parametric generator with a non-parametric retriever, conditioning generation on retrieved passages [Lewis et al., 2020]. This decouples factual knowledge storage from the generator’s parameters, improving provenance and updatability, but the original formulation retrieves a fixed number of passages for every input.

2.2 Adaptive and Active Retrieval

Self-RAG [Asai et al., 2024] trains a single generator LM to emit special reflection tokens that control whether to retrieve, and to critique the relevance and support of retrieved passages, enabling the model to adaptively retrieve on-demand or skip retrieval entirely. This requires fine-tuning the generator and training a separate critic model to produce training supervision.

FLARE [Jiang et al., 2023] operates at a different granularity: during long-form generation, it anticipates the next sentence, checks whether the anticipated content contains low-confidence tokens, and if so retrieves and regenerates. This targets long-form, knowledge-intensive generation rather than short-form factoid or multi-hop QA.

Adaptive-RAG [Jeong et al., 2024] is the closest prior work to our setting: a smaller classifier LM is trained to predict query complexity from automatically collected labels, routing each query between no-retrieval, single-step retrieval, and iterative multi-step retrieval strategies. Our controller shares the three-way action space but differs in (a) using retrieval-time and generation-time signals rather than a purely query-side complexity classifier, and (b) explicitly avoiding fine-tuning of the generator or the complexity model beyond a small external classifier.

Corrective RAG (CRAG) [Yan et al., 2024] always performs an initial retrieval, then uses a lightweight evaluator to assign a confidence degree to the retrieved documents, triggering one of three corrective actions – refine-and-use, discard-and-web-search, or a blended ambiguous strategy. Because CRAG’s evaluator only acts *after* the first retrieval, it cannot realize the efficiency gains of skipping retrieval altogether – a gap our controller is explicitly designed to address via a pre-retrieval gate.

Mallen et al. [Mallen et al., 2023] introduced PopQA and showed that entity popularity is a strong proxy for whether parametric knowledge suffices, motivating a simple popularity-threshold retrieval policy. We use PopQA as a dataset precisely because it provides this kind of external difficulty signal for validating our controller’s decisions (Section 5.1).

2.3 RAG Evaluation

RAGAS [Es et al., 2024] introduced reference-free metrics for RAG pipelines – context relevance, faithfulness (factual consistency of the answer with retrieved context), and answer quality – without requiring ground-truth human annotation. We adopt RAGAS-style faithfulness scoring as our primary automated hallucination proxy (Section 5.3), while acknowledging its known limitations as an LLM-judged metric (Section 10).

2.4 Positioning

Table 1 summarizes how our proposed controller relates to the closest prior methods along the dimensions that matter for our research question.

3 Problem Formulation

Let q be a query, D a fixed corpus, R a retriever such that $D_q = R(q, k)$ returns the top- k documents for q , and G a frozen (not fine-tuned) generator LLM. A controller C maps a query to a discrete retrieval action:

$$C(q) \rightarrow a \in \{\text{NO_RETRIEVAL}, \text{RETRIEVE}, \text{RETRIEVE_MORE}\} \quad (1)$$

¹FLARE always performs an initial retrieval before its first generation step; it does not have a pure no-retrieval branch.

Method	Skips 1st retrieval?	Fine-tunes generator?	Iterative retrieval?	Reports cost
Standard RAG	No	No	No	No
Self-RAG [Asai et al., 2024]	Yes	Yes	Yes	Secondary
FLARE [Jiang et al., 2023]	Partial ¹	No	Yes (mid-generation)	Secondary
CRAG [Yan et al., 2024]	No	No	No (web-search fallback)	Secondary
Adaptive-RAG [Jeong et al., 2024]	Yes	No (classifier only)	Yes	Secondary
Ours	Yes	No	Yes	Primary

Table 1: Positioning relative to closest prior adaptive/active RAG methods. “Primary” cost reporting means efficiency is evaluated with the same statistical rigor (CIs, paired tests) as accuracy/faithfulness, rather than reported as an average without significance testing.

Stage 1 features (computed before any retrieval, so evaluating them costs nothing beyond the query itself):

$$x_1(q) = [\text{len}(q), \text{entity_count}(q), \text{question_type}(q)] \quad (2)$$

Stage 2 features (computed only when Stage 1 is ambiguous, after a first retrieval D_q):

$$x_2(q, D_q) = [\text{conf}_{\text{ret}}(D_q, q), \text{ev}(D_q, q), \text{conf}_{\text{ans}}(G(q, D_q))] \quad (3)$$

where $\text{conf}_{\text{ret}}(D_q, q) = \text{sim}(\text{embed}(q), \text{embed}(d_1))$ is the top-1 retrieval similarity, $\text{ev}(D_q, q)$ is an evidence-coverage score (Section 5.3), and conf_{ans} is the generator’s answer confidence, estimated via self-consistency across sampled generations.

Learned Stage-2 classifier (used only on the subset of queries Stage 1 marks ambiguous):

$$P(a \mid x_1, x_2) = \text{softmax}(W \cdot [x_1; x_2] + b) \quad (4)$$

Target label collection. For a labeled training split, we run all three actions for each query offline and record the action that maximizes a cost-weighted quality objective:

$$y(q) = \arg \max_a [\text{quality}(a, q) - \lambda \cdot \text{cost}(a)] \quad (5)$$

where quality combines answer correctness and faithfulness, cost combines retrieval calls and tokens, and λ is a tunable trade-off weight (Section 5). *The exact value(s) of λ to report, and whether results are sensitive to it, must be experimentally determined – we treat λ as a swept hyperparameter, not a fixed assumption.*

Training loss for the Stage-2 classifier is standard cross-entropy against labels $y(q)$ from Eq. 5:

$$\mathcal{L}(W, b) = -\frac{1}{N} \sum_{i=1}^N \log P(y(q_i) \mid x_1(q_i), x_2(q_i, D_{q_i})) \quad (6)$$

4 Proposed Method: The Adaptive Retrieval Controller

4.1 Architecture

Figure 1 shows the full pipeline. The design deliberately keeps every component external to the generator: no component requires backpropagating through G .

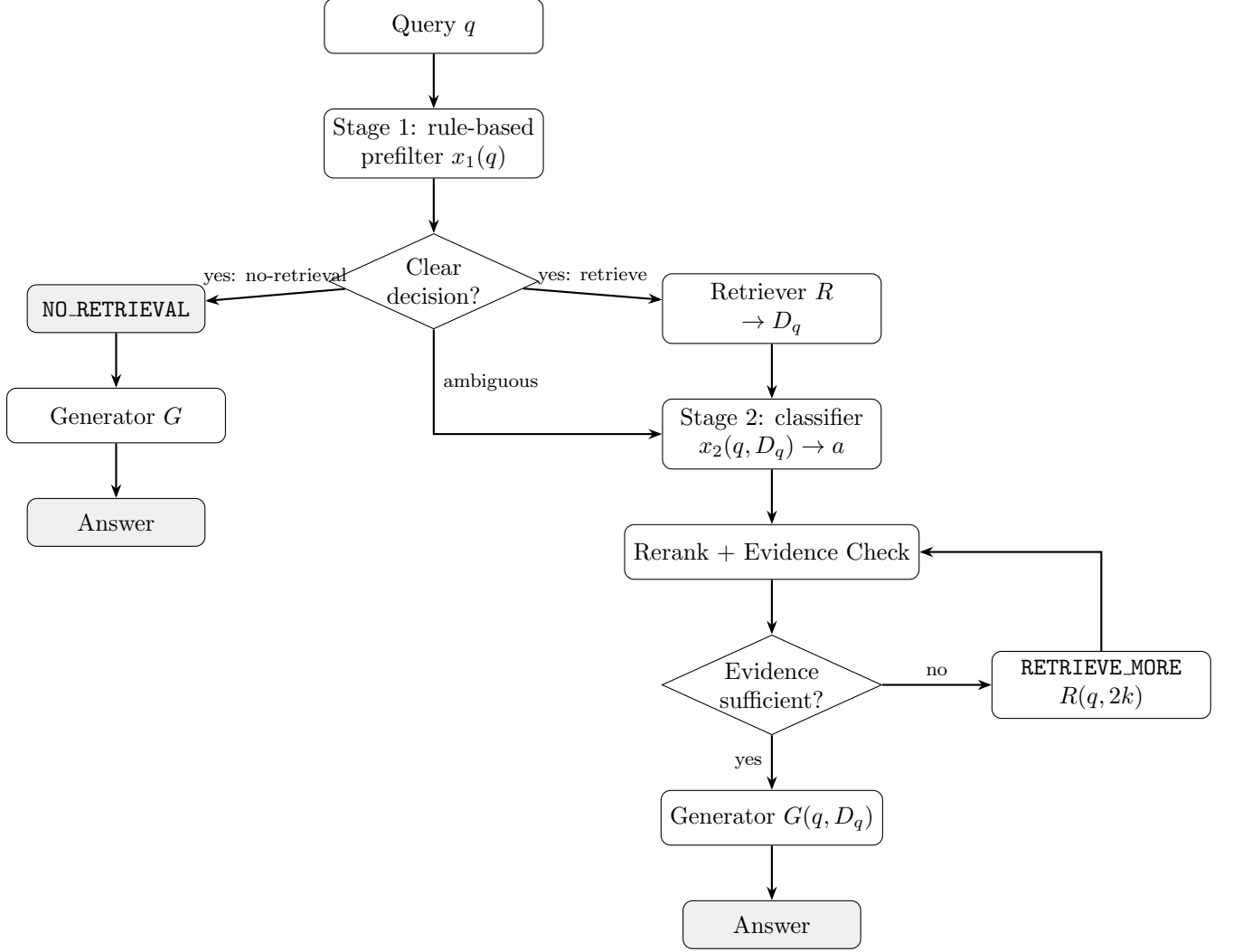


Figure 1: Adaptive retrieval controller architecture. Stage 1 is a near-zero-cost rule-based prefilter computed before any retrieval; Stage 2 (a small trained classifier) only runs on queries Stage 1 cannot confidently route. The RETRIEVE_MORE loop is bounded (Section 4) to guarantee termination.

4.2 Inference procedure

Algorithm 1 details the inference-time procedure, including the termination bound on iterative retrieval (max one RETRIEVE_MORE step, to keep worst-case cost bounded and comparable to Adaptive-RAG’s iterative mode).

4.3 Why not a fine-tuned or per-query LLM-call controller

We deliberately rule out two alternative controller designs. First, a fully learned neural controller sharing weights with, or fine-tuned jointly with, the generator (Self-RAG’s approach) is infeasible on our target hardware (a single free-tier GPU). Second, an LLM-call-per-decision controller – asking the generator itself, via prompting, whether to retrieve – adds non-trivial latency/cost overhead to *every* query, which directly undermines the efficiency objective we are optimizing for. Both remain valid upper-bound comparisons and are included as optional, clearly-labeled reference points if

Algorithm 1 Adaptive Retrieval Controller – Inference

Require: query q , retriever R , generator G , trained classifier f_θ , thresholds $\tau_{\text{low}}, \tau_{\text{high}}$

```
1:  $x_1 \leftarrow \text{EXTRACTSTAGE1FEATURES}(q)$ 
2: if RULESCORE( $x_1$ )  $< \tau_{\text{low}}$  then
3:   return  $G(q)$                                      {confidently answerable without retrieval}
4: else if RULESCORE( $x_1$ )  $> \tau_{\text{high}}$  then
5:    $a \leftarrow \text{RETRIEVE}$ 
6: else
7:    $D_q \leftarrow R(q, k)$ 
8:    $x_2 \leftarrow \text{EXTRACTSTAGE2FEATURES}(q, D_q)$ 
9:    $a \leftarrow \arg \max_a f_\theta(x_1, x_2)$ 
10: end if
11:  $D_q \leftarrow R(q, k)$                                {if not already retrieved above}
12:  $D_q \leftarrow \text{RERANK}(D_q, q)$ 
13: if EVIDENCECOVERAGE( $D_q, q$ )  $< \text{threshold}$  and not already retried then
14:    $D_q \leftarrow D_q \cup R(q, 2k)$                    {single bounded RETRIEVE.MORE step}
15: end if
16: return  $G(q, D_q)$ 
```

compute allows (Section 5.2).

5 Experimental Setup

5.1 Datasets

- **PopQA** [Mallen et al., 2023]: single-hop, entity-centric QA with a built-in popularity-based difficulty signal, directly probing the NO_RETRIEVAL vs. RETRIEVE decision (H1).
- **HotpotQA, distractor setting** [Yang et al., 2018]: multi-hop QA with a small (10-paragraph) per-question candidate pool, probing the RETRIEVE_MORE decision (H3) without requiring a large hosted vector index – a deliberate concession to free-tier compute.
- **TriviaQA** [Joshi et al., 2017] (optional): used as a sanity-check dataset with questions often answerable from parametric knowledge alone, serving as a negative control for over-triggering retrieval.

Exact subsample sizes per dataset, and the train/validation/test split used for Stage-2 classifier training, are to be finalized based on measured per-query compute cost on the target hardware (Section 5.4) and reported here once fixed.

5.2 Baselines

- **No-RAG**: $G(q)$ directly, no retrieval.
- **Standard RAG**: fixed top- k retrieval for every query, $G(q, R(q, k))$.
- **Adaptive-RAG** [Jeong et al., 2024]: reproduced from the authors’ released implementation where feasible, or reimplemented from the paper’s description if adaptation to our

datasets/generator is required. *Which of these applies must be stated explicitly once implementation begins, per the project’s baseline-transparency requirement.*

- **CRAG** [Yan et al., 2024]: reproduced analogously, with the web-search-fallback component either disabled (closed-corpus setting, to keep comparisons fair) or clearly flagged as enabled if used.
- **Ours**: the controller in Section 4.

Self-RAG and FLARE are discussed as related work (Section 2) but are not included as primary reproduced baselines, since fair reproduction would require either fine-tuning a generator (Self-RAG) or a long-form generation setting mismatched to our short-form QA datasets (FLARE); both are noted as directions for future comparison rather than silently omitted.

5.3 Metrics

Answer quality: Exact Match (EM), F1. **Faithfulness / hallucination:** RAGAS-style [Es et al., 2024] claim-level groundedness of the generated answer against retrieved context, reported as a faithfulness rate; hallucination is operationalized as the complement (ungrounded claims per answer), not as an unmeasured qualitative impression. **Retrieval quality:** Recall@ k , evidence-coverage rate. **Efficiency:** retrieval calls per query, input/output tokens, wall-clock latency, and an estimated \$-cost per 1,000 queries computed from the generator’s published per-token pricing (or, if a local open-weight model is used, from measured GPU-time). **Statistical protocol:** bootstrap 95% confidence intervals for every reported metric, and paired significance tests (e.g., paired bootstrap or Wilcoxon signed-rank on per-query scores) between our controller and each baseline, per the project’s Section 13 statistical-analysis requirement.

5.4 Compute setup

All conditions use a single frozen generator LLM (*model to be finalized based on what fits a free-tier T4 GPU with quantization – placeholder, not yet fixed*) and a single fixed sentence-embedding model for retrieval, held constant across all baselines and the proposed controller so that observed differences are attributable to the controller’s retrieval *decisions*, not to generator or retriever differences (holding this constant is a controlled-variable requirement carried over from Section 9 of the project brief).

6 Results

This section is a structural placeholder. No experiments have been run yet. Table 2 shows the reporting format that will be populated once the protocol in Section 5 has actually been executed. This must be experimentally validated.

7 Ablation Study

Placeholder, pending experiments. Planned ablations, directly testing which components of the controller matter (H4):

- **A1:** Stage 2 without conf_{ret} (retrieval confidence).

Method	EM	F1	Faithfulness	Retrieval calls/query	Tokens/query	Latency (s)
No-RAG	–	–	–	–	–	–
Standard RAG	–	–	–	–	–	–
Adaptive-RAG (repro.)	–	–	–	–	–	–
CRAG (repro.)	–	–	–	–	–	–
Ours	–	–	–	–	–	–

Table 2: *Placeholder. All cells to be filled with mean $\pm$ bootstrap 95% CI after running the protocol in Section 5. Aggregated across PopQA + HotpotQA (distractor).*

- **A2:** Stage 2 without conf_{ans} (answer confidence).
- **A3:** Stage 2 without $\text{ev}(D_q, q)$ (evidence coverage).
- **A4:** Stage 1 rule-based filter alone, Stage 2 disabled entirely (tests H4 directly: does the cheap filter capture most of the gain?).
- **A5:** RETRIEVE_MORE disabled (single-pass retrieval only).
- **A6:** Fixed top- k vs. adaptive k .

Configuration	EM	Faithfulness	Retrieval calls/query
Full controller	–	–	–
A1: – conf_{ret}	–	–	–
A2: – conf_{ans}	–	–	–
A3: – ev	–	–	–
A4: rule-only (Stage 1)	–	–	–
A5: no RETRIEVE_MORE	–	–	–

Table 3: *Placeholder ablation table.*

8 Error Analysis

Placeholder, pending experiments. Once results are available, failures will be manually categorized (per the project’s Section 14 error taxonomy) into: incorrect no-retrieval decisions, unnecessary retrieval, retrieval failure despite a correct RETRIEVE decision, poor or conflicting evidence, hallucination despite correct evidence, and multi-hop reasoning failures on HotpotQA specifically. Representative examples will be included for each category, with a discussion of which failure modes are attributable to the controller’s decision versus the underlying retriever/generator.

9 Discussion

This section cannot be written meaningfully before Sections 6–8 are complete. Once populated, it should directly address H1–H4 from the project brief: whether the controller matched/beat Standard RAG’s quality while reducing retrieval volume (H1); whether it reduced hallucination on knowledge-intensive questions relative to No-RAG (H2); whether it reduced latency/tokens relative to always-on RAG (H3); and which signals (Section 7) were actually load-bearing (H4).

10 Limitations

- **Compute scale:** all experiments target free-tier single-GPU (Colab T4) compute, which constrains dataset subsample sizes, generator model size, and corpus size (we use the HotpotQA distractor setting rather than full-Wikipedia open-domain retrieval specifically for this reason). Findings may not transfer to larger-scale deployments.
- **No generator fine-tuning:** by design, this excludes the potentially higher ceiling that fine-tuned approaches like Self-RAG can reach; our comparison is explicitly about what a non-fine-tuning controller can achieve, not an unconditional claim of superiority.
- **Closed corpus:** unlike CRAG, we do not use live web search as a fallback, which keeps the corpus reproducible but caps the maximum achievable evidence quality for genuinely under-covered queries.
- **Faithfulness metric limitations:** RAGAS-style automated faithfulness scoring is itself an imperfect, LLM-judged proxy with known biases; results should be read with this caveat, and, where feasible, spot-checked against human judgment.
- **Single/few generator backbone(s):** results may not generalize across generator model families or sizes; this is left as future work.
- **Statistical power:** small subsampled test sets (a consequence of the compute constraint above) limit the power of significance tests, particularly for the ablation study’s finer-grained comparisons.

11 Conclusion

We have posed a specific, falsifiable version of the adaptive-retrieval question: whether a lightweight, non-fine-tuning controller combining retrieval-confidence, evidence-coverage, and answer-confidence signals can match or improve on existing adaptive RAG methods’ quality-faithfulness-cost trade-off. We presented the controller’s design, mathematical formulation, and a pre-registered evaluation protocol with statistical rigor applied equally to accuracy, faithfulness, and efficiency. *Whether the hypothesis holds is, deliberately, not claimed here – it is the empirical question the experimental protocol in Section 5 is designed to answer, and results will be added to this manuscript only after that protocol has actually been run.*

References

- Akari Asai, Zeqiu Wu, Yizhong Wang, Avirup Sil, and Hannaneh Hajishirzi. Self-RAG: Learning to retrieve, generate, and critique through self-reflection. In *International Conference on Learning Representations (ICLR)*, 2024.
- Shahul Es, Jithin James, Luis Espinosa-Anke, and Steven Schockaert. RAGAs: Automated evaluation of retrieval augmented generation. In *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations (EACL)*, pages 150–158. Association for Computational Linguistics, 2024.
- Soyeong Jeong, Jinheon Baek, Sukmin Cho, Sung Ju Hwang, and Jong C. Park. Adaptive-RAG: Learning to adapt retrieval-augmented large language models through question complexity. In

- Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 7036–7050, Mexico City, Mexico, 2024. Association for Computational Linguistics.
- Zhengbao Jiang, Frank F. Xu, Luyu Gao, Zhiqing Sun, Qian Liu, Jane Dwivedi-Yu, Yiming Yang, Jamie Callan, and Graham Neubig. Active retrieval augmented generation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 7969–7992, Singapore, 2023. Association for Computational Linguistics.
- Mandar Joshi, Eunsol Choi, Daniel S. Weld, and Luke Zettlemoyer. TriviaQA: A large scale distantly supervised challenge dataset for reading comprehension. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1601–1611, Vancouver, Canada, 2017. Association for Computational Linguistics.
- Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, 2020.
- Alex Mallen, Akari Asai, Victor Zhong, Rajarshi Das, Daniel Khashabi, and Hannaneh Hajishirzi. When not to trust language models: Investigating effectiveness of parametric and non-parametric memories. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Toronto, Canada, 2023. Association for Computational Linguistics.
- Shi-Qi Yan, Jia-Chen Gu, Yun Zhu, and Zhen-Hua Ling. Corrective retrieval augmented generation. *arXiv preprint arXiv:2401.15884*, 2024.
- Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W. Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. HotpotQA: A dataset for diverse, explainable multi-hop question answering. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 2369–2380, Brussels, Belgium, 2018. Association for Computational Linguistics.